Store Inventory and Multi-Batch Pricing System

Write a command-line program in C that runs a small store: it imports stock, sells through shopping carts, tracks costs with FIFO batches, applies a promotion and gifts, and saves its data to text files.

At a Glance

- **Deadline:** December 11, 11:59 p.m. Late submissions are not accepted.
- **Submit:** the link to your public GitHub repo, on E3 (Section 5.2).
- **Demo:** planned for the TA session on December 15.
- **Grading:** public tests 30%, hidden tests 30%, report 20%, Demo 20%.
- **Start:** unzip the resource package, read README.md, build with make and run `./checker`.

1. Requirements

1.1 Implementation Rules

- **Language:** C and the C standard library only (for example stdio.h, stdlib.h, string.h, ctype.h, math.h). No C++.
- **Files:** at least 2 .c files and 1 .h file, and each .c file has its own job. main calls functions in the other .c files through declarations in a .h file; do not #include a .c file. A main.c that only reads commands and calls the other files is fine.
- **Linked lists:** store the products, and the batches of each product, in linked lists allocated with malloc, not in arrays. Other data structures are up to you.
- **Memory:** remove and free a batch as soon as it is used up, and free all memory before the program ends. Appendix B shows how to remove nodes safely.

1.2 Building and Running

- Develop on any system; Linux is recommended. On Windows, use Code::Blocks or VS Code with MinGW (gcc and mingw32-make).
- The TA team builds your project on Linux x86-64 with gcc and GNU make, using your Makefile. Start from the provided template and edit only the SRCS and HDRS lines.
- Build and run in the project root folder (on Windows: `mingw32-make` and `inventory.exe`). File names in commands are relative to this folder.

```
make
./inventory < input.txt > output.txt
```

- File names are case-sensitive on Linux: `#include "Store.h"` does not find store.h.
- Do not use conio.h, windows.h, system("pause"), getch, itoa, fflush(stdin), strdup, strtok_r, strcasecmp, stricmp or strlwr. They may work on Windows but fail on Linux. Fix every "implicit declaration" warning.

1.3 Input

Item	Rule
Product / cart ID	Letters or digits, 1–15 characters; case-sensitive.
Product name	Letters, digits or spaces, 1–31 characters. A name may start or end with spaces, or consist only of spaces, and must be kept exactly as given. The same product ID always has the same name.
Selling price, unit cost	Whole dollars, 1–10,000.

Item	Rule
Batch quantity, cart item quantity	1–1,000; for a cart item, the limit applies to the total after several ADD commands.
Promotion parameters	N: 1–100; P: 1–99, meaning P% of the original price.
Maximum counts	At most 100 products, 1 current promotion and 10 open carts at the same time.
Batch ID	Shared by the whole system, increasing from 1, at most 1,000,000; the next batch ID is at most 1,000,001.
Quantities and amounts	Total quantities, stock values, transaction and accumulated amounts, and intermediate values of formulas all fit in a 32-bit integer; profit can be negative.
Line length	Each line of commands, stock files and save files has at most 128 characters (not counting the newline); file names contain no spaces.

- One command per line; arguments are separated by spaces. The last command is always QUIT.
- Input is always valid: command names, argument counts, formats and values follow the table above. For PROMO, A and B exist and are different; CART never reuses the ID of an open cart; a cart has at least one item when it is checked out.
- At start there are no products, carts or promotion, and all totals are 0.
- Print only the output described in this document: no menus, prompts or debug messages.
- IDs are compared in ASCII order.

1.4 Checking Your Program

- **checker.c**, included in the resource package, runs your program on the 10 public tests and prints PASS or FAIL for each. For a failing test, it shows the first wrong line and the command that printed it.
- Build your program with make, then build the checker once and run it in the project folder. After each change, run make and `./checker` again:

```
gcc checker.c -o checker
./checker
```

- checker.c has its own main and is not part of your program: do not list it in SRCS. See README.md for details.

2. Commands and Files

2.1 Commands

pid and cid are product and cart IDs. Replace the values in angle brackets.

Command	Function	Output on success
`IMPORT <file>`	Imports a stock file and creates products as needed. New products have no price yet.	`OK IMPORT <batches>`
`PRICE <pid> <price>`	Sets or updates the selling price of a product.	`OK PRICE <pid> <price>`
`PROMO <A> <N> <B> <P>`	Sets the promotion: when the quantity of A is at least N, B is sold at P% of its price. A new promotion replaces the old one.	`OK PROMO`
`CART <cid>`	Creates an empty cart.	`OK CART <cid>`

Command	Function	Output on success
`ADD <cid> <pid> <qty>`	Adds a product to the cart, adding up quantities. Does not check price or stock and does not reserve stock.	`OK ADD <cid> <pid> <qty>`
`CHECKOUT <cid>`	Checks out as described in Section 3. An existing cart is closed whether checkout succeeds or fails.	A receipt; see Section 4.
`QUERY <pid>`	Shows the price, stock and batches of a product.	Query result; see Section 4.
`REPORT`	Shows the stock value and the accumulated profit and loss.	Stock and profit report; see Section 4.
`SAVE <file>`	Saves the system data to the given file. Carts are not saved.	`OK SAVE`
`LOAD <file>`	Replaces the current data with the data in the file. On success, all carts are closed.	`OK LOAD`
`QUIT`	Frees all dynamic memory and ends the program; main must return 0.	`BYE`

- IMPORT prints the number of batches imported. ADD prints the total quantity of that product in the cart.
- A product stays in the catalog after it sells out.

2.2 Stock File

Each line of a stock file is one batch:

```
<pid>###<name>@@@<qty>&&&<unit_cost>
P001###Coffee Beans@@@2&&&100
```

- Lines are never merged, even for the same product and cost. Batch IDs are global and given in import order: file by file, line by line.
- A stock file has at least one line and no empty lines; the last line may lack a newline.
- Importing adds stock only; it does not change revenue or costs.
- **Tip:** `strstr` from string.h finds ###, @@@ and &&&. Names may contain spaces, so `%s` cannot read them.

2.3 Saving and Loading

- **SAVE** writes all data except carts in the format of Appendix A: products (including sold-out ones) with prices, the promotion, remaining batches, totals and the next batch ID. It creates or overwrites the file, changes no data, and prints OK SAVE.
- **LOAD** replaces all data with the data in the file and closes all carts. Later imports continue from the loaded next batch ID.
- If IMPORT, SAVE or LOAD cannot open the file, print ERR FILE_OPEN and change nothing. Files given to LOAD are always complete and valid.
- **Tip:** fgets keeps the newline, so read a name as a whole line and remove the newline. LOAD reads as many records as each header says. Save values, not pointers; LOAD creates new nodes and keeps batch IDs and FIFO order.

3. Checkout and Errors

3.1 Checkout

CHECKOUT uses the prices, promotion and stock at the moment of checkout, and closes the cart whether it succeeds or fails. All carts share the same stock. It works in this order:

- **1. Check.** Go through the items in product ID order. For each item, check that it has a price, then that there is enough stock. At the first error, print it and stop: no stock or totals change.
- **2. Price.** Compute each line's subtotal with integer division (this rounds to the nearest dollar):

```
subtotal = (price * qty * P + 50) / 100
```

  P is the promotion percent if the line is product B and the cart has at least N of product A; otherwise P = 100. TOTAL is the sum of all subtotals.

- **3. Take stock (FIFO).** Take each product from its oldest batch first, moving on to later batches as needed. COST is the sum of (quantity taken × unit cost of that batch).
- **4. Gifts.** The customer gets 1 gift per full 1,000 dollars of TOTAL. Choose each gift again among the products that have a price and stock: the smallest "price − unit cost of its oldest batch" wins (it can be negative); on a tie, the smallest ID wins. A gift takes 1 unit by FIFO, and its cost is added to the gift cost.

  Gifts add no revenue, do not count toward the promotion, and do not earn more gifts. If no product can be chosen, stop giving gifts; the checkout still succeeds.

- **5. Totals.** PROFIT = TOTAL − COST − gift cost. Add TOTAL, COST and the gift cost to the accumulated revenue, cost of goods sold and gift cost.

3.2 Errors

A failed command prints one line, ERR <code>, and changes nothing, except that CHECKOUT still closes the cart. Then continue with the next command.

Error output	When
`ERR NO_PRODUCT`	The product given to PRICE, ADD or QUERY is not in the catalog.
`ERR NO_CART`	The cart given to ADD or CHECKOUT does not exist or is closed.
`ERR NO_PRICE`	CHECKOUT finds a purchased product without a price.
`ERR INSUFFICIENT <pid>`	CHECKOUT finds a product without enough stock.
`ERR FILE_OPEN`	IMPORT or LOAD cannot open the file, or SAVE cannot create or open the file.

- ADD checks the cart before the product. CHECKOUT reports the first error found in step 1.
- QUERY on a product with no stock is not an error: it prints stock=0.

4. Output Formats

- Replace <...> with the actual values. Print numbers as plain integers, without thousands separators.
- Every line ends with a newline. Do not print blank lines.
- Item lines and batch lines start with 2 spaces.

4.1 Receipt

```
RECEIPT <cid>
  <pid> x<qty> @<price> = <subtotal>
TOTAL <total>
COST <cost>
<gift_lines>
PROFIT <profit>
```

- One item line per product, in product ID order; price is the original price. The discounted line (product B when the promotion applies) looks like this:

```
  <pid> x<qty> @<price> *<P>% = <subtotal>
```

- Do not print *100% on lines without a discount. COST covers only the purchased items, not the gifts.
- Gift lines go between COST and PROFIT:

Case	Output
At least 1 gift given	`One line GIFT <pid> @<unit_cost> per gift, in the order given; gifts of the same product are not merged.`
No gift given	`Print GIFT none.`
Some gifts not given	`After the lines above, also print GIFT_SHORT <count>; print nothing when no gift is missing.`

- In a GIFT line, unit_cost is the batch cost of that gift.

4.2 Query

```
QUERY <pid> <name> price=<price> stock=<stock>
  #<batch_id> qty=<qty> cost=<unit_cost>
```

- price=none if the product has no price.
- One line per remaining batch, in FIFO order; qty is what is left in that batch. With no stock, print only the first line (stock=0).

4.3 Report

```
REPORT
  <pid> stock=<stock> value=<value>
REVENUE <revenue>
COGS <cogs>
GIFTCOST <gift_cost>
PROFIT <profit>
```

- List only products with stock greater than 0, in ID order (including products without a price). value is the sum of qty × unit cost over the remaining batches.
- REVENUE, COGS and GIFTCOST are the accumulated totals; PROFIT = REVENUE − COGS − GIFTCOST. Before any sale they are all 0.

5. Example, Submission and Grading

5.1 Example

Starting from an empty system, with this stock.txt:

```
P001###Coffee Beans@@@2&&&100
P001###Coffee Beans@@@4&&&120
P002###Mug@@@4&&&150
```

Standard input	Standard output
IMPORT stock.txt PRICE P001 300 PRICE P002 250 PROMO P001 3 P002 80 CART C1 ADD C1 P001 3 ADD C1 P002 1 CHECKOUT C1 REPORT QUIT	OK IMPORT 3 OK PRICE P001 300 OK PRICE P002 250 OK PROMO OK CART C1 OK ADD C1 P001 3 OK ADD C1 P002 1 RECEIPT C1 P001 x3 @300 = 900 P002 x1 @250 *80% = 200 TOTAL 1100 COST 470 GIFT P002 @150 PROFIT 480 REPORT P001 stock=3 value=360 P002 stock=2 value=300 REVENUE 1100 COGS 470 GIFTCOST 150 PROFIT 480 BYE

P001 costs 2×100 + 1×120 = 320 and P002 costs 150, so COST = 470. TOTAL 1,100 earns 1 gift; P002 has the smaller "price − batch cost", so it is the gift. PROFIT = 1,100 − 470 − 150 = 480.

5.2 Submission

- Put your project in a **public** GitHub repo and submit the link on E3 by **December 11, 11:59 p.m.** Late submissions are not accepted.
- The repo root must contain all your .c and .h files, the Makefile and report.pdf. You do not need to include checker.c, tests/, runs/ or .exe files; checker.c is not part of your program and must not be listed in SRCS.
- **report.pdf:** your program structure, main design, test results, and the problems you met and how you fixed them. If you used AI, say what for and how much, and what you adopted, changed and verified; otherwise say that you did not use AI.

5.3 Grading

Item	Weight	What is assessed
Public tests	30%	Basic features and basic error handling, with the given commands and output formats.
Hidden tests	30%	Features, edge cases and the error handling of Section 3.2, including the order of checks, the state after failures, file open failures, loading given save files, and checking SAVE files.
Report	20%	Design description, testing and fixes, and AI use.
Demo	20%	Using the program, understanding the code, design reasons, and handling rule changes.

- There are 10 public and 10 hidden tests, 3 points each. A test passes only if all standard output and every saved file match exactly.
- The TA team builds your project with make on Linux and runs each test with its input.txt as standard input. A crash, or a non-zero return value from main, fails the test. Make sure your project builds with make.
- Each violation of the rules in Section 1.1 costs 10 points. Passing the tests does not mean the rules are met.

- In the Demo, explain your code, including the job of each .c file and the declarations in your .h files. You may look at your code. Schedule details will be announced later.

Appendix A: Save File Format

- SAVE and LOAD use this format. The sections always appear in this order, with upper-case keywords; <...> marks a value.
- Each product name is on its own line and keeps its spaces. Other fields are separated by one space.
- Numbers are plain integers. SAVE ends every line with a newline and writes no blank lines; the last line of a file given to LOAD may lack a newline.

A.1 Sections

Format (written from top to bottom)	Content and order
`NEXT_BATCH <next_batch_id>`	The global batch ID that the next import will use. Save and restore it as it is; do not compute it from the largest remaining batch ID.

`PRODUCTS <product_count>`
`<pid> <price>`
`<name>`
	First the number of products, then two lines per product in increasing product ID order: the ID and price, then the full name. Includes products with no stock; price is -1 for a product without a price.

`PROMO <promo_count>` `<A> <N> <B> <P>`	promo_count is 0 or 1. With 0, the promotion line is omitted; with 1, one line with the current promotion follows.
`TOTALS <revenue> <cogs> <gift_cost>`	Accumulated revenue, cost of goods sold and gift cost, in this order; always one line.
`BATCHES <batch_count>` `<pid> <batch_id> <qty> <unit_cost>`	First the total number of remaining batches, then one line per batch: product ID, batch ID, remaining quantity and unit cost. In increasing product ID order, and in FIFO order within a product; only batches with a remaining quantity greater than 0 are saved.

- All five section headers are always written; a count of 0 means that no records follow. Carts and profit are not saved.

A.2 Examples

SAVE inventory.txt before QUIT in the example of Section 5.1 writes:

```
NEXT_BATCH 4
PRODUCTS 2
P001 300
Coffee Beans
P002 250
Mug
PROMO 1
P001 3 P002 80
TOTALS 1100 470 150
BATCHES 2
P001 2 3 120
P002 3 2 150
```

Right after the program starts, the save file is:

```
NEXT_BATCH 1
PRODUCTS 0
PROMO 0
TOTALS 0 0 0
BATCHES 0
```

Appendix B: Removing and Freeing Linked List Nodes

This standalone example uses printers and their print jobs. Each printer keeps its first (head) and last (tail) job. It shows how to remove the first node, why tail must become NULL when the only node is removed, and how to free inner nodes before outer ones. The batch logic of this project is yours to design.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Job {
    int id, pages;
    struct Job *next;
} Job;

typedef struct Printer {
    char name[16];
    Job *head, *tail;               /* first and last job */
    struct Printer *next;
} Printer;

/* Append a job at the end of the printer's queue. */
void add_job(Printer *p, int id, int pages) {
    Job *job = malloc(sizeof *job);
    if (job == NULL) exit(1);
    job->id = id;
    job->pages = pages;
    job->next = NULL;
    if (p->tail == NULL) p->head = job;   /* queue was empty */
    else p->tail->next = job;
    p->tail = job;
}

/* Remove and free the first job. */
void finish_first_job(Printer *p) {
    Job *job = p->head;
    if (job == NULL) return;
    p->head = job->next;
    if (p->head == NULL) p->tail = NULL;  /* it was the only job */
    free(job);
}

/* Free every job of every printer, then the printers. */
void free_printers(Printer *list) {
    while (list != NULL) {
        Printer *next_printer = list->next;
        while (list->head != NULL) finish_first_job(list);
        free(list);
        list = next_printer;
    }
}
```

Using the functions; each comment shows the output of that line:

```
void print_queue(const Printer *p) {
    const Job *job;
    printf("%s:", p->name);
    for (job = p->head; job != NULL; job = job->next)
        printf(" #%d(%d)", job->id, job->pages);
    printf("\n");
}

int main(void) {
```

```
    Printer *lab = calloc(1, sizeof *lab);   /* head = tail = next = NULL */
    if (lab == NULL) return 1;
    strcpy(lab->name, "Lab");
    add_job(lab, 1, 5);
    add_job(lab, 2, 3);
    print_queue(lab);          /* Lab: #1(5) #2(3) */
    finish_first_job(lab);     /* removes the head */
    finish_first_job(lab);     /* removes the only job: tail becomes NULL */
    print_queue(lab);          /* Lab: */
    add_job(lab, 3, 8);        /* works because tail was reset */
    print_queue(lab);          /* Lab: #3(8) */
    free_printers(lab);
    return 0;
}
```

If tail were not set to NULL, it would point to freed memory, and the next add_job would write there and could crash the program.